

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
LABORATORY OBSERVATION & RECORD

BASIC INFORMATION

Student Name: P. Harsha Vardhan

Roll No: A24126552103

Branch & Sec: CSM-B

Date of Execution: 04/09/2026

Course Name: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 02

Title: Creating an Interactive To-Do List Application using HTML, CSS, and JavaScript

OBJECTIVE / AIM

To design and implement an interactive To-Do List web application featuring dynamic DOM manipulation, task status toggling, deletion, and local storage data persistence using HTML, CSS, and JavaScript.

SYSTEM REQUIREMENTS

Hardware Requirements:

Processor (Intel Core i5 / AMD Ryzen 5), 8GB RAM, Storage.

Software Requirements:

Operating System: Windows 11 / Linux

Code Editor: Visual Studio Code

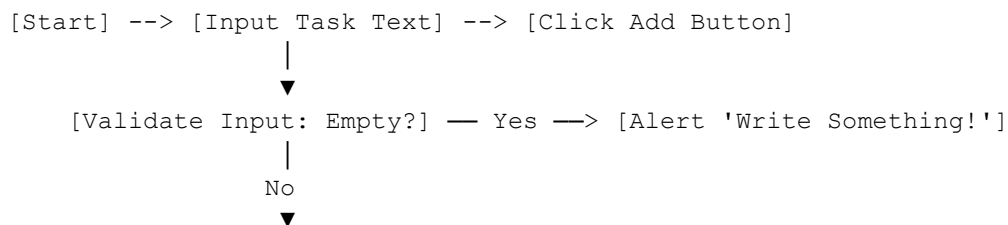
Browser: Google Chrome / Mozilla Firefox

ARCHITECTURE / DESIGN

1. System Architecture Block:

- Frontend UI Layer: Structured using HTML5 (index.html) and styled using CSS3 (style.css) with linear gradient backgrounds and responsive card layouts.
- Application Logic & State: JavaScript (script.js) manages event listeners, dynamic DOM element creation/deletion, and browser localStorage persistence.

2. Execution Process Flow:



[Create & Deletion] —> [Append to list-container]



[Save HTML to LocalStorage] <— [Toggle Checked / Remove Task]

3. Execution Steps Summary:

Step 1: Initialize HTML container and input row structure linked with CSS and JavaScript.

Step 2: Capture input field value on 'Add' button click; validate that string is non-empty.

Step 3: Dynamically append list item () with delete button () to target container.

Step 4: Attach click event listener to toggle task completion status or delete task element.

Step 5: Persist list DOM tree state across reloads using localStorage.setItem() and showTask().

SOURCE CODE

HTML (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do-List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <div class="todo-app">
      <h2>To-Do List </h2>
      <div class="row">
        <input type="text" id="input-box" placeholder="Add your text">
        <button onclick="addTask()">Add</button>
      </div>
      <ul id="list-container"></ul>
    </div>
  </div>
</body>
<script src="script.js"></script>
</html>
```

CSS (style.css):

```
*{
  margin: 0;
  padding: 0;
  font-family: "Poppins", sans-serif;
  box-sizing: border-box;
}
.container{
  width: 100%;
  min-height: 100vh;
```

```

        background: linear-gradient(135deg, #153677, #4e085f);
        padding: 10px;
    }
    .todo-app{
        width: 100%;
        max-width: 540px;
        background: white;
        margin: 100px auto 20px;
        padding: 40px 30px 70px;
        border-radius: 10px;
    }
    .todo-app h2{
        color: #002765;
        display: flex;
        align-items: center;
        margin-bottom: 20px;
    }
    .todo-app h2 img{
        width: 30px;
        margin-left: 10px;
    }
    .row{
        display: flex;
        align-items: center;
        justify-content: space-between;
        background: #edeef0;
        border-radius: 30px;
        padding-left: 20px;
        margin-bottom: 25px;
    }
    input{
        flex: 1;
        border: none;
        outline: none;
        background: transparent;
        padding: 10px;
        font-weight: 14px;
    }
    button{
        border: none;
        outline: none;
        padding: 16px 50px;
        background: #ff5945;
        color: #fff;
        font-size: 16px;
        cursor: pointer;
        border-radius: 40px;
    }
    ul li{
        list-style: none;
        font-size: 17px;
        padding: 12px 8px 12px 50px;
    }

```

```

        user-select: none;
        cursor: pointer;
        position: relative;
    }
    ul li::before{
        content: "";
        position: absolute;
        height: 28px;
        width: 28px;
        border-radius: 50%;
        background-image: url(images/unchecked.png);
        background-size: cover;
        background-position: center;
        top: 12px;
        left: 8px;
    }
    ul li.checked{
        color: #555;
        text-decoration: line-through;
    }
    ul li.checked::before{
        background-image: url(images/checked.png);
    }
    ul li span{
        position: absolute;
        right: 0;
        top: 5px;
        width: 40px;
        height: 40px;
        font-size: 22px;
        color: #555;
        line-height: 40px;
        text-align: center;
        border-radius: 50%;
    }
    ul li span:hover{
        background: #edeef0;
    }

```

JavaScript (script.js):

```

const inputBox = document.getElementById("input-box");
const listContainer = document.getElementById("list-container");

function addTask(){
    if(inputBox.value === ''){
        alert("You Must Write Something!");
    }
    else{
        let li = document.createElement("li");
        li.innerHTML = inputBox.value;
        listContainer.appendChild(li);
        let span = document.createElement("span");
    }
}

```

```

        span.innerHTML = "\u00d7";
        li.appendChild(span);
    }
    inputBox.value = "";
    saveData();
}

listContainer.addEventListener("click", function(e) {
    if(e.target.tagName === "LI") {
        e.target.classList.toggle("checked");
        saveData();
    }
    else if(e.target.tagName === "SPAN") {
        e.target.parentElement.remove();
        saveData();
    }
}, false);

function saveData() {
    localStorage.setItem("data", listContainer.innerHTML);
}
function showTask() {
    listContainer.innerHTML = localStorage.getItem("data");
}
showTask();

```

COMPILATION / EXECUTION STEPS

Step 1: Create a project directory containing index.html, style.css, script.js, and an images/ folder.

Step 2: Ensure image assets (icon.png, unchecked.png, checked.png) are placed inside the images directory.

Step 3: Open index.html directly using any modern web browser or start Live Server in Visual Studio Code.

Step 4: Enter task text into the input field and click the 'Add' button to create a task.

Step 5: Click on any task item to toggle its completed state, or click the 'x' button to delete it.

Step 6: Refresh the page to verify that localStorage preserves task data automatically.

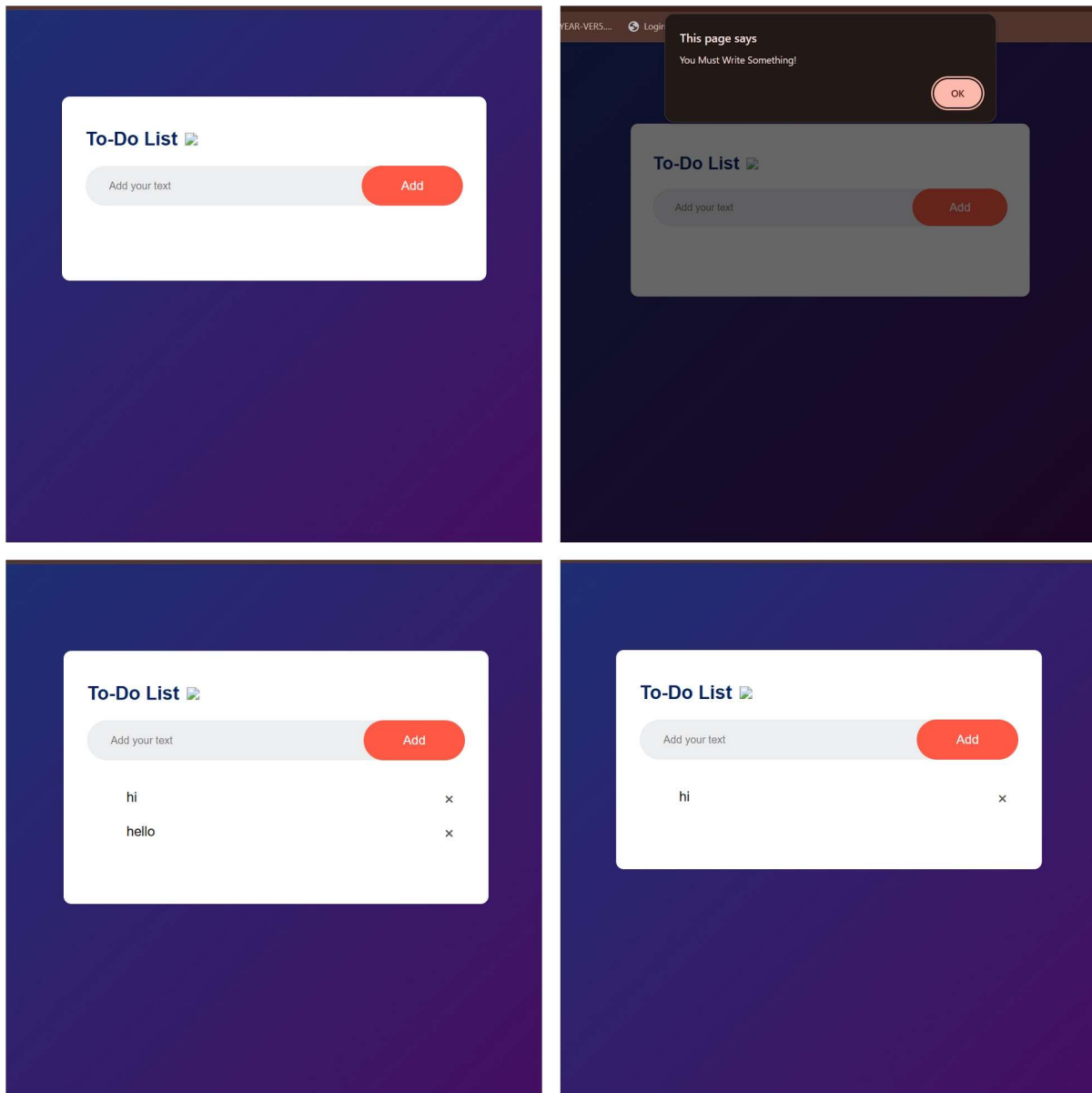
EXPECTED OUTPUT / SCREENSHOTS

Test Cases & Verification Results:

Test ID	Test Input / Action	Expected Output	Status
TC-01	Click 'Add' with empty input box	Alert popup 'You Must Write Something!' displayed	Passed
TC-02	Enter text and click 'Add' button	Task appended to list with delete icon	Passed

TC-03	Click on added task list item	Toggles strikethrough class 'checked' and checked icon	Passed
TC-04	Click 'x' span on task item	Task item removed permanently from list	Passed
TC-05	Refresh / reload browser page	Saved list items rendered from localStorage	Passed

OUTPUT:



RESULT / CONCLUSION

The interactive To-Do List application was successfully implemented, tested, and verified using HTML5, CSS3, and JavaScript. Task addition, deletion, completion toggling, and localStorage data persistence functioned as expected.